TraceOpt

Choosing compiler optimizations by zero-knowledge proving cost: a demo report with real RISC Zero measurements.

16	1	0	15/16
evaluation cases (4 kernels × 4 inputs)	proving-cost savings taken (up to 50%)	cases where TraceOpt made proving worse	proved sizes predicted exactly

1. The idea in plain words

A **zero-knowledge virtual machine** (zkVM) such as RISC Zero runs a program and produces a **proof** that it ran correctly. Anyone can check the proof quickly without re-running the program. Producing the proof is the expensive part, and its cost grows with the number of execution steps (**cycles**).

RISC Zero proves cycles in blocks whose size is a **power of two**: a run that needs 140,000 cycles is proved as 262,144. Saving cycles therefore only lowers the proving cost when the run drops into a smaller power-of-two size. Ordinary compilers optimize for CPU speed and know nothing about this.

TraceOpt generates several versions of a program (here: loop unrolling), predicts the proving size of each one *without running the zkVM*, and chooses the cheapest version it can trust. The zkVM then runs the chosen version so the prediction can be checked against the real numbers.

Pipeline

C program
a small kernel with an input `n`

→

LLVM
clang + opt build versions unrolled 1×, 2×, 4×, 8×

→

Predict
count executed operations, apply the fitted cost model, round up to a power of two

→

Decide
cheapest version that is a safe saving and passes answer checks

→

RISC Zero
runs the version; real answer and cycle counts are compared with the prediction

2. Example: `strided_rw` with n = 5

A memory-heavy kernel just above a power-of-two boundary.

The program (`benchmarks/strided_rw.c`)

```
// Memory-intensive kernel: fills a 64 KiB static array, then performs n
// passes of strided read-modify-write (1 KiB stride, 64 accesses per pass).
// Intended to touch many distinct memory pages in the zkVM.
#include <stdint.h>

#define WORDS 16384u /* 64 KiB of uint32_t */
#define STRIDE 256u  /* 1 KiB */

static uint32_t buf[WORDS];

uint32_t traceopt_kernel(uint32_t n) {
  for (uint32_t i = 0; i < WORDS; i++) {
    buf[i] = i * 2654435761u;
  }
  uint32_t acc = 0;
  for (uint32_t r = 0; r < n; r++) {
    for (uint32_t i = 0; i < WORDS; i += STRIDE) {
      uint32_t k = (i + r) & (WORDS - 1);
      acc = (acc ^ buf[k]) + r;
      buf[k] = acc;
    }
  }
  return acc;
}
```

Input	n = 5 passes over a 64 KiB array
Output	a 32-bit checksum of the values written
Expected answer	**3528213632** (computed independently by the Python reference in tests/reference.py)

Versions TraceOpt considers

Unrolling copies the loop body so each round of the loop does several iterations' work: fewer compare-and-jump steps run, but the code gets bigger. Each version is run in TraceOpt's IR interpreter on several inputs and must return the same answer as the original (a test on those inputs, not a proof of equivalence).

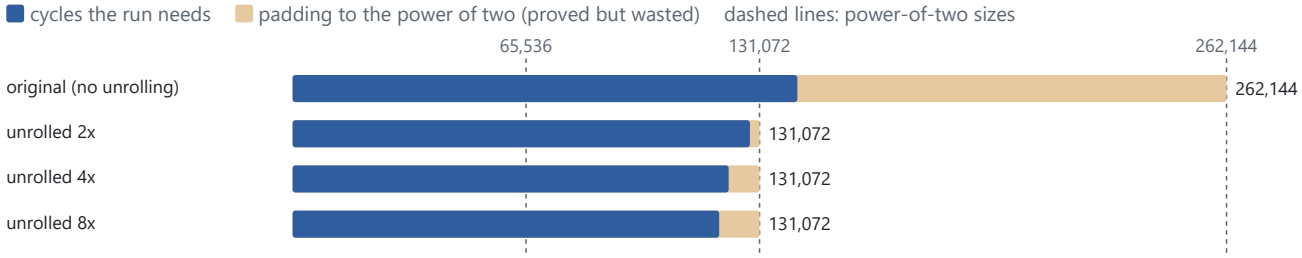
version	code size	IR instructions	compute ops executed	memory ops executed	control ops executed	same answer?
original (no unrolling)	140 B	30	67,787	17,024	16,717	reference
unrolled 2x	420 B	109	59,438	17,024	8,369	yes, on 10 inputs
unrolled 4x	1,160 B	252	55,264	17,024	4,193	yes, on 10 inputs
unrolled 8x	2,944 B	712	53,185	17,024	2,106	yes, on 10 inputs

Predicted cost

needed = program cycles (from the model) + paging cycles (loading and saving the 1 KiB memory pages the program touches) + 4,113 reserved cycles. The prover charges the next power of two.

version	program	paging	needed	proved as
original (no unrolling)	76,083	61,475	141,671	**262,144**
unrolled 2x	62,786	61,475	128,374	**131,072**
unrolled 4x	56,136	62,147	122,396	**131,072**

version	program	paging	needed	proved as
unrolled 8x	52,819	62,819	119,751	**131,072**



Decision

A version is chosen only if it lowers the proved size by at least 5%, still does so when its prediction is 10% too optimistic, passed the answer checks, and grows the code by at most 16×.

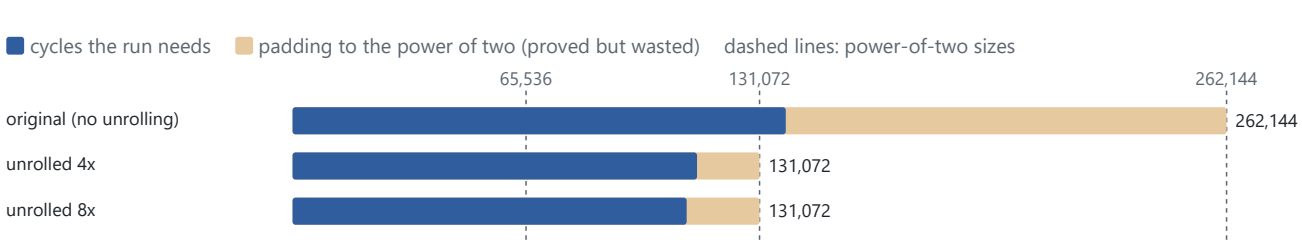
	version	why
–	original (no unrolling)	the program as written; every other version is compared with it
✗	unrolled 2x	smaller bucket only if the prediction is accurate; a 10% prediction error would undo it: rejected as too risky
✓ chosen	unrolled 4x	uses a smaller bucket even if the prediction is pessimistic: accepted
✗	unrolled 8x	code grows to 2,944 bytes, more than 16x the original: rejected

TraceOpt chooses unrolled 4x: predicted proved size 131,072 instead of 262,144 (50% less proving work).

Checked in the real RISC Zero zkVM

Each listed version was linked into the RISC Zero guest and executed with the same input. Measured are the original, the chosen version, and the rejected version with the best prediction (the one a greedy optimizer would have picked).

version	zkVM answer	correct?	program cycles predicted	measured	proved as predicted	measured
original (no unrolling)	3528213632	yes	76,083	73,180	262,144	**262,144**
unrolled 4x **(chosen)**	3528213632	yes	56,136	48,031	131,072	**131,072**
unrolled 8x	3528213632	yes	52,819	43,797	131,072	**131,072**



Measured proved size: 262,144 → 131,072 (50% less proving work).

unrolled 8x (rejected) measured 131,072: no better than the choice.

3. Example: `xorshift` with n = 1000

Unrolling saves cycles, but whether it saves proving work is uncertain.

The program (`benchmarks/xorshift.c`)

```
// Simple loop with a loop-carried dependence: n rounds of xorshift32.
// Unlike sum_squares, LLVM cannot replace this loop with a closed form,
// so its cycle count scales with n.
#include <stdint.h>

uint32_t traceopt_kernel(uint32_t n) {
  uint32_t x = 2463534242u;
  for (uint32_t i = 0; i < n; i++) {
    x ^= x << 13;
    x ^= x >> 17;
    x ^= x << 5;
  }
  return x;
}
```

Input	n = 1000 rounds of a random-number generator
Output	the generator's 32-bit state after n rounds
Expected answer	**3298996588** (computed independently by the Python reference in tests/reference.py)

Versions TraceOpt considers

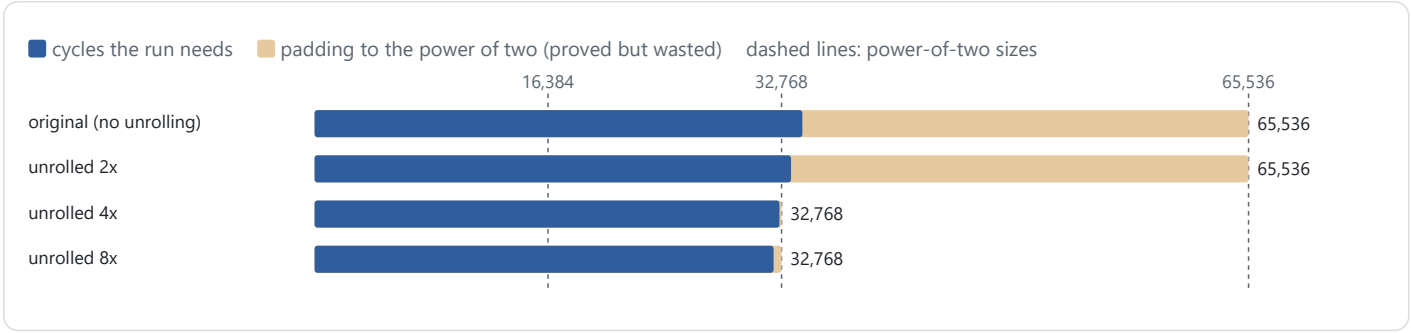
Unrolling copies the loop body so each round of the loop does several iterations' work: fewer compare-and-jump steps run, but the code gets bigger. Each version is run in TraceOpt's IR interpreter on several inputs and must return the same answer as the original (a test on those inputs, not a proof of equivalence).

version	code size	IR instructions	compute ops executed	memory ops executed	control ops executed	same answer?
original (no unrolling)	52 B	15	8,001	0	1,002	reference
unrolled 2x	136 B	46	7,506	0	507	yes, on 11 inputs
unrolled 4x	216 B	69	6,756	0	257	yes, on 11 inputs
unrolled 8x	312 B	93	6,381	0	132	yes, on 11 inputs

Predicted cost

needed = program cycles (from the model) + paging cycles (loading and saving the 1 KiB memory pages the program touches) + 4,113 reserved cycles. The prover charges the next power of two.

version	program	paging	needed	proved as
original (no unrolling)	11,646	18,467	34,226	**65,536**
unrolled 2x	10,858	18,467	33,438	**65,536**
unrolled 4x	10,035	18,467	32,615	**32,768**
unrolled 8x	9,623	18,467	32,203	**32,768**



Decision

A version is chosen only if it lowers the proved size by at least 5%, still does so when its prediction is 10% too optimistic, passed the answer checks, and grows the code by at most 16×.

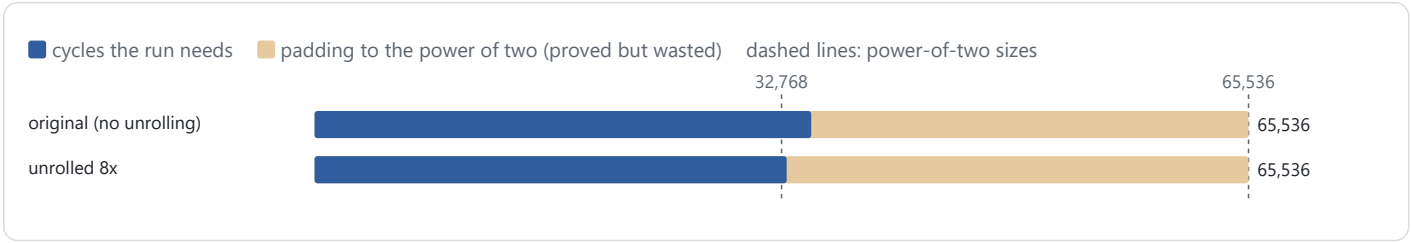
	version	why
✓ chosen	original (no unrolling)	the program as written; every other version is compared with it
✗	unrolled 2x	fewer cycles, but still rounded up to the same bucket: no saving
✗	unrolled 4x	smaller bucket only if the prediction is accurate; a 10% prediction error would undo it: rejected as too risky
✗	unrolled 8x	smaller bucket only if the prediction is accurate; a 10% prediction error would undo it: rejected as too risky

TraceOpt keeps the original program: no version is a safe saving.

Checked in the real RISC Zero zkVM

Each listed version was linked into the RISC Zero guest and executed with the same input. Measured are the original, the chosen version, and the rejected version with the best prediction (the one a greedy optimizer would have picked).

version	zkVM answer	correct?	program cycles predicted	measured	proved as predicted	measured
original (no unrolling) **(chosen)**	3298996588	yes	11,646	12,099	65,536	**65,536**
unrolled 8x	3298996588	yes	9,623	10,356	32,768	**65,536**



unrolled 8x (rejected) was predicted to need 32,768 but really needed 65,536: the promised saving did not happen, so rejecting it was right.

4. Full evaluation

4 kernels × 4 inputs (n = 5, 50, 300, 3000), none of them used to fit the model. Each build was executed in RISC Zero and its output checked against the reference. Values are the measured proved size (padded cycles); lower is better. "Best possible" is the lowest any unroll version reached.

kernel	n	original (-O1, no unroll)	clang -O2	TraceOpt	best possible	outcome
`xorshift`	5	32,768	32,768	32,768 (original)	32,768	no gain possible
`xorshift`	50	32,768	32,768	32,768 (original)	32,768	no gain possible
`xorshift`	300	32,768	32,768	32,768 (original)	32,768	no gain possible
`xorshift`	3000	65,536	65,536	65,536 (original)	65,536	no gain possible
`nested_fnv`	5	32,768	32,768	32,768 (original)	32,768	no gain possible
`nested_fnv`	50	32,768	32,768	32,768 (original)	32,768	no gain possible
`nested_fnv`	300	65,536	65,536	65,536 (original)	65,536	no gain possible
`nested_fnv`	3000	524,288	262,144	524,288 (original)	262,144	win missed
`branchy`	5	32,768	32,768	32,768 (original)	32,768	no gain possible
`branchy`	50	32,768	32,768	32,768 (original)	32,768	no gain possible
`branchy`	300	32,768	32,768	32,768 (original)	32,768	no gain possible
`branchy`	3000	131,072	131,072	131,072 (original)	65,536	win missed
`strided_rw`	5	262,144	262,144	131,072 (unrolled 4x)	131,072	win taken
`strided_rw`	50	262,144	262,144	262,144 (original)	262,144	no gain possible
`strided_rw`	300	524,288	524,288	524,288 (original)	524,288	no gain possible
`strided_rw`	3000	2,621,440	2,621,440	2,621,440 (original)	2,097,152	win missed

outcome	cases
No version could lower the proved size; original kept	12
A saving was available and TraceOpt took it	1
A saving was available and TraceOpt missed it	3
TraceOpt's choice was worse than the original	0
Predicted proved size of TraceOpt's choice was exactly right	15 of 16

How accurate is the cycle model?

Leave-one-kernel-out test: the model is fitted without a kernel, then predicts that kernel. Mean and worst relative error of predicted program cycles:

kernel left out	mean error	worst error
`branchy`	1.5%	3.5%
`nested_fnv`	6.4%	14.4%
`strided_rw`	18.3%	37.1%
`xorshift`	7.1%	17.8%

5. What this shows, and its limits

- Proving cost moves in power-of-two steps, so most cycle savings do not change it. An optimizer needs to know where the steps are.
- TraceOpt predicts cycles from LLVM IR without running the zkVM and uses the prediction to decide; when it changed the code, the predicted saving was real.
- It is deliberately cautious near the step edges: that avoided false wins and also missed some real ones.

Limits: four small kernels; one optimization (loop unrolling); cycle counts from RISC Zero's executor (full proofs did not fit in the development machine's memory); answers checked on test inputs, which is not a formal proof of equivalence.

Reproduce

```
wsl -d Ubuntu
cd /mnt/c/ ... /TraceOpt
scripts/build_backend.sh          # once, about 4 minutes
scripts/demo.sh                   # narrated terminal demo
scripts/demo.sh --no-pause --json results/demo-run.json
python3 scripts/make_report.py    # rebuilds this page
```

Model: `results/model-clang21.json`. Program cycles ≈ 92 + 0.851 × compute + 0.113 × memory + 0.742 × control + 4005; paging ≈ 18467 + 672 × pages touched.